

Relatório

- Daniel Wesley Freitas Siqueira GRR20245621
- Gabriel Gioia de Brito GRR20235159

1. Algoritmos

Os algoritmos desenvolvidos para a criptografia foram feitos com paralelismo de operações em mente.

Cada iteração do algoritmo trabalha sobre blocos de 512b, e cada bloco é considerado como 4 números de 128b.

Algorithm 1: Encryption

```
1: procedure DGENC_ENCRYPT(text, key, result)
2:   mask  $\leftarrow$  generateMask(key)
3:   reduced_mask  $\leftarrow$  reduceAdd(key)
4:
5:   for  $i \leftarrow 0$  to text.len - 1 do
6:     xored  $\leftarrow$  text[i]  $\oplus$  mask
7:     reduced  $\leftarrow$  reduceAdd(xored)
8:     red_xor_mask  $\leftarrow$  reduced  $\oplus$  reduced_mask
9:     rem  $\leftarrow$  red_xor_mask mod 4
10:    result[i][0]  $\leftarrow$  xored[rem mod 4]
11:    result[i][1]  $\leftarrow$  xored[(rem + 1) mod 4]
12:    result[i][2]  $\leftarrow$  xored[(rem + 2) mod 4]
13:    result[i][3]  $\leftarrow$  xored[(rem + 3) mod 4]
14:   end
15: end
```

Algorithm 2: Decryption

```
1: procedure DGENC_DECRYPT(text, key, result)
2:   mask  $\leftarrow$  generateMask(key)
3:   reduced_mask  $\leftarrow$  reduceAdd(key)
4:
5:   for  $i \leftarrow 0$  to text.len - 1 do
6:     reduced  $\leftarrow$  reduceAdd(text[i])
7:     red_xor_mask  $\leftarrow$  reduced  $\oplus$  reduced_mask
8:     rem  $\leftarrow$  red_xor_mask mod 4
9:     res[0]  $\leftarrow$  xored[rem mod 4]
10:    res[1]  $\leftarrow$  xored[(rem + 1) mod 4]
11:    res[2]  $\leftarrow$  xored[(rem + 2) mod 4]
12:    res[3]  $\leftarrow$  xored[(rem + 3) mod 4]
13:    xored  $\leftarrow$  res  $\oplus$  mask
14:    result[i]  $\leftarrow$  xored
15:   end
16: end
```

- **generateKey** é um acúmulo sucessivo dos blocos de 512b da chave com operações XOR.
- **reduceAdd** é um acúmulo sucessivo de blocos de 128b com operações ADD.

Os algoritmos de cifra e decifra consideram que a mensagem tem um tamanho múltiplo do bloco com que eles trabalham.

2. Implementação

Foram feitas duas implementações dos algoritmos: uma escrita em zig utilizando *builtins* para otimizações com instruções vetoriais, e outra em C com *intrinsics* de AVX512 diretamente.

- A compilação do código em C foi feita com `zig cc -c -mavx512f -mavx512bw -O3 vectorized_dgenc.c -o vectorized_dgenc.o`
- A compilação do código em zig após a geração do arquivo objeto anterior foi feita com `zig build -Doptimize=ReleaseFast -Dcpu=native`.

3. Execução

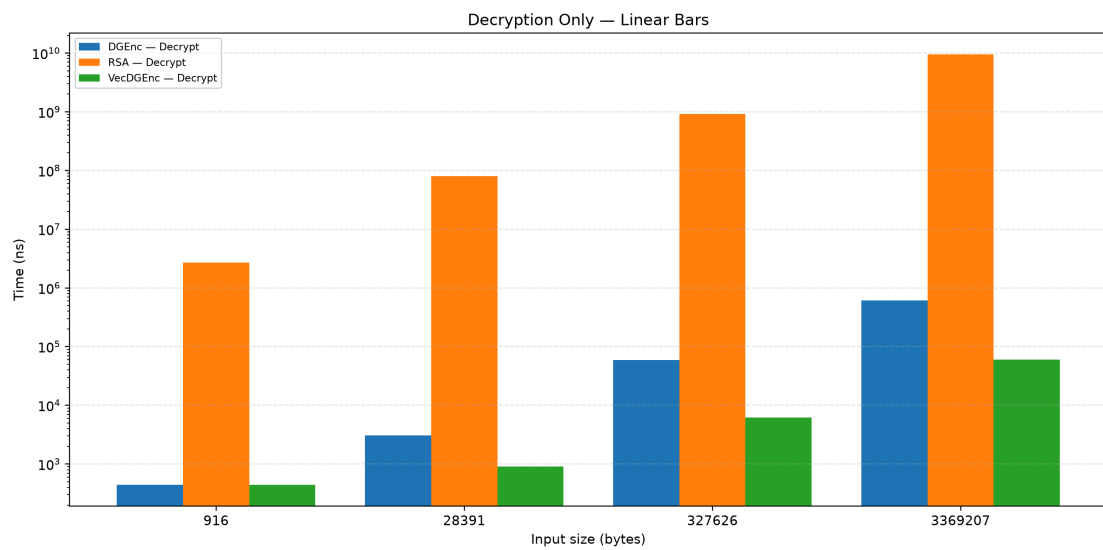
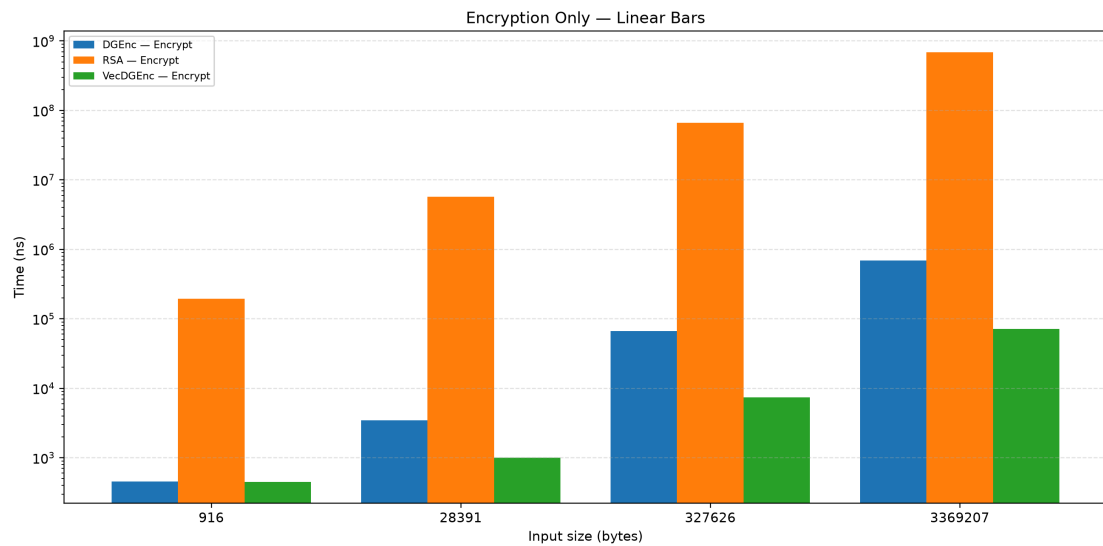
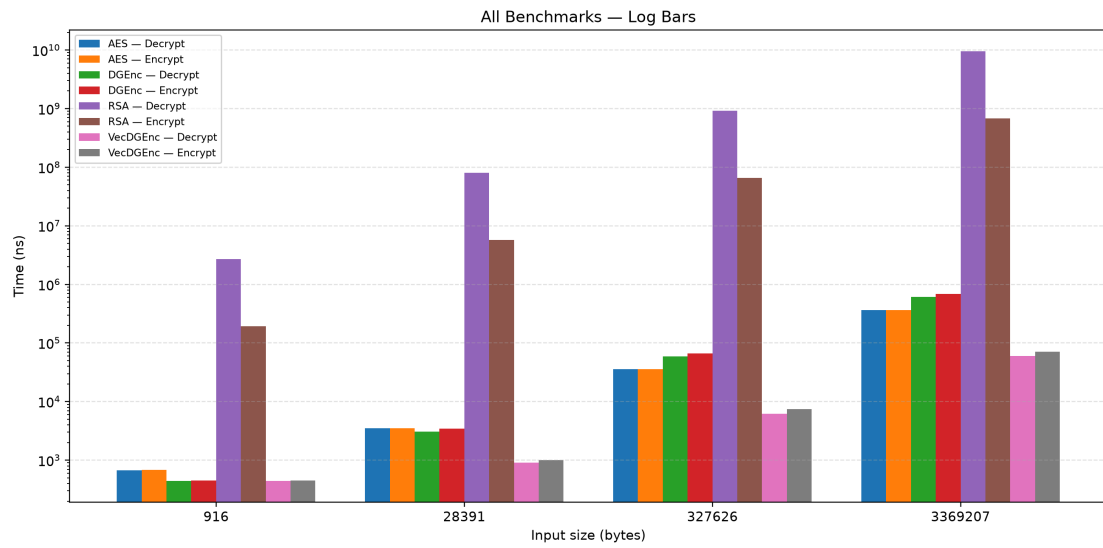
- Tempos de cifra e decifra foram avaliados separadamente
 1. 1 execução sem medições para evitar anomalias com cache
 2. Mediu-se o tempo de 30 execuções e o resultado foi a média aritmética

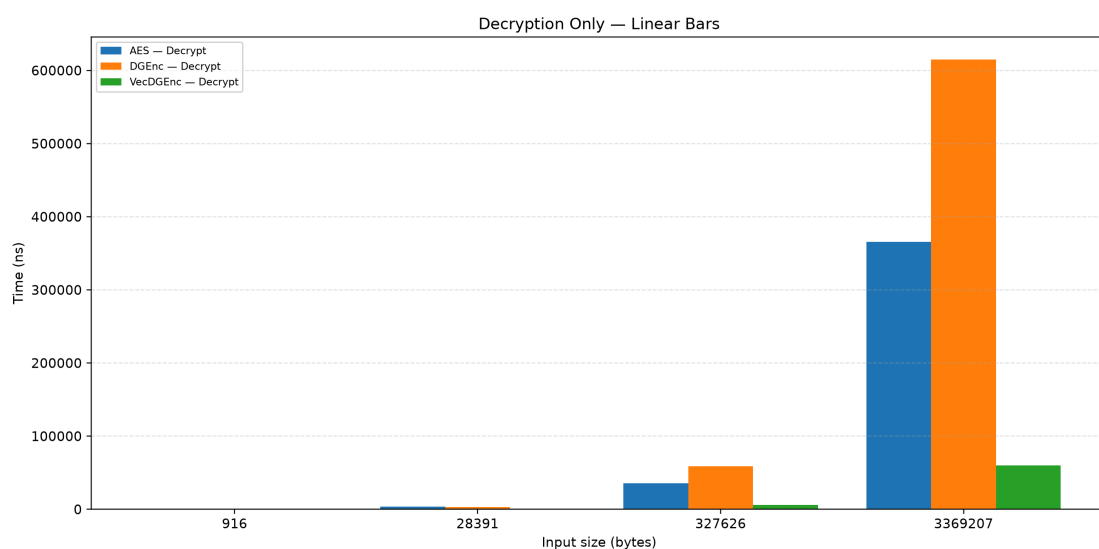
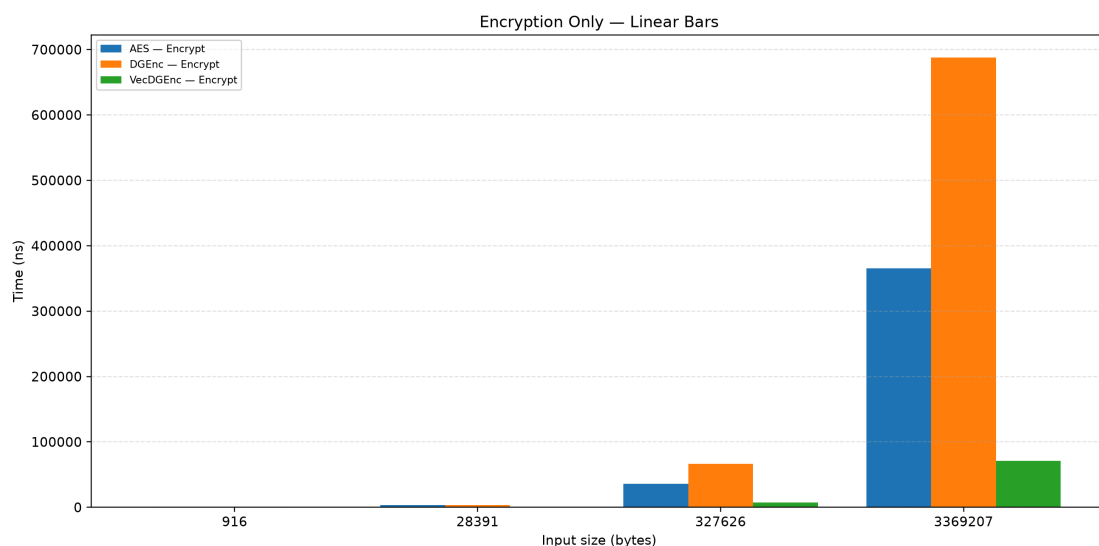
4. Considerações

É esperado que a execução do algoritmo desenvolvido seja bastante eficiente, já que:

- realiza apenas operações rápidas para a CPU (ADD e XOR e resto mod 4) por laço de repetição,
- passa apenas uma vez pelos dados, o que evita cache misses
- tira proveito de bastante paralelismo com instruções vetoriais (XOR e shuffle).

5. Resultados





6. Especificações

Os testes foram executados em uma máquina com as seguintes especificações:

Architecture:	x86_64
CPU op-mode(s):	32-bit, 64-bit
Address sizes:	48 bits physical, 48 bits virtual
Byte Order:	Little Endian
CPU(s):	12
On-line CPU(s) list:	0-11
Vendor ID:	AuthenticAMD
Model name:	AMD Ryzen 5 8500G w/ Radeon 740M
Graphics	
CPU family:	25

```

Model: 120
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
Stepping: 0
Microcode version: 0xa70800a
Frequency boost: enabled
CPU(s) scaling MHz: 60%
CPU max MHz: 5080,2632
CPU min MHz: 414,7160
BogoMIPS: 7087,07
Flags: fpu vme de pse tsc msr pae mce
cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse
sse2 ht syscall nx mmxext fxsr_opt pdpe1gb rdtscp lm
constant_tsc re
                                p_good amd_lbr_v2 nopl xtopology
nonstop_tsc cpuid extd_apicid aperfmperf rapl pni pclmulqdq
monitor ssse3 fma cx16 sse4_1 sse4_2 movbe popcnt aes xsave
avx f16c
                                rdrand lahf_lm cmp_legacy svm
extapic cr8_legacy abm sse4a misalignsse 3dnowprefetch osvw
ibs skinit wdt tce topoext perfctr_core perfctr_nb bpext
perfctr_llc m
                                waitx cpuid_fault cpb cat_l3
cdp_l3 hw_pstate ssbd mba perfmon_v2 ibrs ibpb stibp
ibrs_enhanced vmmcall fsgsbase bmi1 avx2 smep bmi2 erms
invpcid cqm rdt_a avx51
                                2f avx512dq rdseed adx smap
avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl
xsaveopt xsavec xgetbv1 xsaves cqm_llc cqm_occup_llc
cqm_mbm_total cqm_m
                                bm_local user_shstk avx512_bf16
clzero irperf xsaveerptr rdpru wbnoinvd cppc arat npt lbrv
svm_lock nrip_save tsc_scale vmcb_clean flushbyasid
decodeassists paus
                                efilter pfthreshold avic vgif
x2avic v_spec_ctrl vnmi avx512vbmi umip pku ospke avx512_vbmi2
gfni vaes vpclmulqdq avx512_vnni avx512_bitalg
avx512_vpopcntdq rdpi
                                d overflow_recov succor smca fsrm
flush_lld amd_lbr_pmc_freeze

```

Virtualization features:

Virtualization: AMD-V

Caches (sum of all):

L1d: 192 KiB (6 instances)

L1i: 192 KiB (6 instances)

L2: 6 MiB (6 instances)

L3: 16 MiB (1 instance)

NUMA:

NUMA node(s): 1

NUMA node0 CPU(s): 0-11

Vulnerabilities:

Gather data sampling: Not affected

Ghostwrite: Not affected

Indirect target selection: Not affected

Itlb multihit: Not affected

L1tf: Not affected

Mds: Not affected

Meltdown: Not affected

Mmio stale data: Not affected

Old microcode: Not affected

Reg file data sampling: Not affected

Retbleed: Not affected

Spec rstack overflow: Mitigation; Safe RET

Spec store bypass: Mitigation; Speculative Store

Bypass disabled via prctl

Spectre v1: Mitigation; usercopy/swapgs

barriers and __user pointer sanitization

Spectre v2: Mitigation; Enhanced / Automatic

IBRS; IBPB conditional; STIBP always-on; PBR SB-eIBRS Not

affected; BHI Not affected

Srbds: Not affected

Tsa: Mitigation; Clear CPU buffers

Tsx async abort: Not affected

Vmscape: Mitigation; IBPB before exit to

userspace

Machine (15GB total)

Package L#0

NUMANode L#0 (P#0 15GB)

L3 L#0 (16MB)

L2 L#0 (1024KB) + L1d L#0 (32KB) + L1i L#0 (32KB) + Core

L#0

```

    PU L#0 (P#0)
    PU L#1 (P#6)
L#1 L2 L#1 (1024KB) + L1d L#1 (32KB) + L1i L#1 (32KB) + Core
    PU L#2 (P#1)
    PU L#3 (P#7)
L#2 L2 L#2 (1024KB) + L1d L#2 (32KB) + L1i L#2 (32KB) + Core
    PU L#4 (P#2)
    PU L#5 (P#8)
L#3 L2 L#3 (1024KB) + L1d L#3 (32KB) + L1i L#3 (32KB) + Core
    PU L#6 (P#3)
    PU L#7 (P#9)
L#4 L2 L#4 (1024KB) + L1d L#4 (32KB) + L1i L#4 (32KB) + Core
    PU L#8 (P#4)
    PU L#9 (P#10)
L#5 L2 L#5 (1024KB) + L1d L#5 (32KB) + L1i L#5 (32KB) + Core
    PU L#10 (P#5)
    PU L#11 (P#11)
HostBridge
  PCIBridge
    PCI 01:00.0 (VGA)
  PCIBridge
    PCIBridge
      PCIBridge
        PCI 07:00.0 (Ethernet)
          Net "enp7s0"
      PCIBridge
        PCI 0c:00.0 (SATA)
          Block(Disk) "sdb"
          Block(Disk) "sda"
)
```